TECHNICAL NOTES - EDVEXA DATA QUALITY & ETL MONITOR

Platform Version: 2.7.0 (Cloud Deployment: Render + Neon PostgreSQL + Neon Object Storage) Author: Raghav Kumar | Software Engineer Project Workspace: c:\Users\rjha1\Downloads\csv_project Live Production URL: https://edvexa-data-quality-etl-monitor.onrender.com <https://edvexa-data-quality-etl-monitor.onrender.com> Live Swagger API Docs: https://edvexa-data-quality-etl-monitor.onrender.com/docs <https://edvexa-data-quality-etl-monitor.onrender.com/docs> Live Health Verification: https://edvexa-data-quality-etl-monitor.onrender.com/api/health <https://edvexa-data-quality-etl-monitor.onrender.com/api/health>

1. Architecture Decisions

1.1 Backend Framework Selection: FastAPI & Uvicorn

FastAPI was selected for this data platform over Django and Flask for several decisive architectural reasons:

1. First-Class Asynchronous Performance & Native Typing: High-throughput file uploads and streaming data validation require non-blocking I/O and strict type safety.
2. Pydantic v2 Schema Enforcement: Request and response schemas are strictly validated at the protocol layer, guaranteeing that malformed query parameters, invalid pagination limits, or unsupported dataset types are intercepted before touching business logic.
3. Automatic OpenAPI/Swagger Generation: Reduces documentation drift by automatically publishing interactive API documentation at /docs reflecting actual Python type annotations.
4. Standard Uvicorn ASGI Runner: Lightweight and cross-platform on Windows with zero binary dependency complications.

1.2 Database Abstraction: SQLAlchemy 2.x

The application employs SQLAlchemy 2.0+ declarative models with standard session management:

* Default Local Store: SQLite (sqlite:///./edvexa_data.db) provides instant zero-setup local execution on Windows without external daemon dependencies.
* Dialect Portability: Model definitions use standard ANSI SQL datatypes (String, Integer, Float, DateTime, Text) with index declarations. The queries utilize modern SQLAlchemy 2.x select() constructs, making the entire data layer drop-in compatible with PostgreSQL (postgresql://...) simply by changing DATABASE_URL in .env.
* Session Scoping: Implemented via a FastAPI dependency generator (get_db()) ensuring transactions are properly committed or rolled back and database connections are safely closed in the finally block.

1.3 Decoupled Service Layer

The codebase adheres strictly to the single responsibility principle:

* app/routers/: Transport layer handling HTTP requests, query parameters, status codes, and JSON serialization.
* app/services/csv_import.py: Ingestion layer responsible for file decoding, charset resolution, structural header inspection, and delimiter handling.
* app/services/transformation.py: Pure, side-effect-free sanitization and normalizers (string trimming, name capitalization, date parsing, numeric casting).
* app/services/validation.py: Business logic validation, email regex compliance, numeric range checks, and error report generation.
* app/services/etl_pipeline.py: Orchestrator executing the transaction boundary, deduplication checks, and metric tallying.

1.4 Frontend Architecture: Lightweight Responsive SPA

The user interface was built using native HTML5, modern CSS3 (custom CSS variables, responsive grid, flexbox layout), Vanilla JavaScript ES6+, and Chart.js via CDN:

* Avoids brittle Node.js build pipelines (npm, webpack, vite), eliminating installation friction for local evaluation on Windows.
* Eliminates client-side framework bloat while providing instant reactive transitions across views (Overview, Import, Jobs, Errors, Records, and Documentation).
* Dynamically queries REST endpoints on every view change, ensuring KPI cards, graphs, and tables always represent the live state of the database.

1.5 Job-Scoped Clean Record Architecture & Excel Export Engine

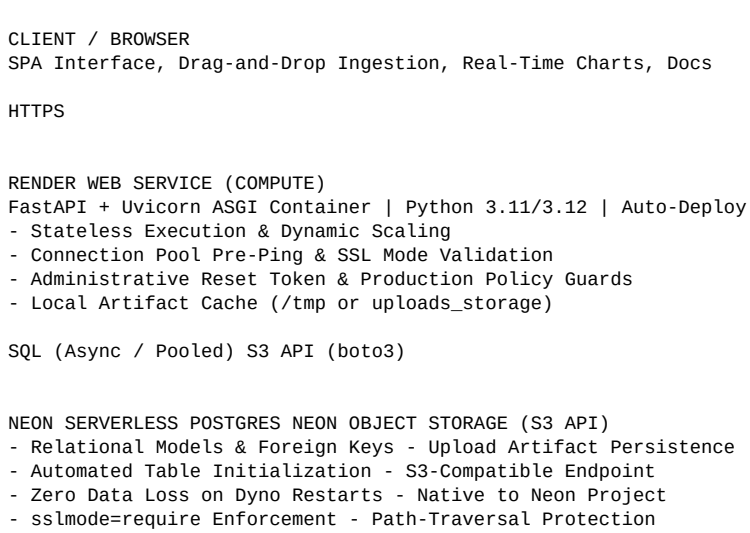
To provide secure, isolated downloads on a per-job basis without corrupting or inferring from global business tables:

1. The Isolation Problem: Clean tables (customers, sales, inventory) maintain global unique deduplication constraints across the lifetime of the platform. A job may accept 5 clean records, but querying the global table would inadvertently export records from other jobs or previous uploads.
2. Dedicated Audit Model (JobCleanRecord): We introduced JobCleanRecord (id, job_id, row_number, dataset_type, business_key, record_json, created_at). During the ETL load phase, every accepted clean record is staged into

job_clean_records in the same atomic database transaction as the business tables.
3. Multi-Sheet Microsoft Excel Workbook Export (generate_job_excel_report): Generates a genuine .xlsx workbook using openpyxl with three dedicated worksheets:
- Sheet 1: Job Summary: Job metadata, execution metrics, error aggregations, and the row accounting rule: total rows = accepted rows + rejected rows + duplicates + skipped rows.
- Sheet 2: Clean Records: Only clean records accepted by that specific job. Features navy blue header rows, frozen panes at A2, auto-filters, numeric formatting (#,##0.00 for currency, #,##0 for quantities), and automated column width fitting.
- Sheet 3: Validation Errors: All validation failures and duplicate detections with row numbers, error codes, error messages, and original values. Features frozen header row (A2), auto-filters, and text wrapping.
4. Empty State Resilience: The Excel workbook can always be generated for completed jobs even if accepted rows or validation errors equal 0, rendering a clean explanatory note instead of erroring out.
5. Deterministic CSV Generation: generate_clean_records_csv and generate_validation_errors_csv stream isolated single-purpose CSV files.
6. Zero-Downtime Backwards-Compatibility: For pre-existing historical jobs processed before JobCleanRecord was introduced, a fallback resolver (backfill_job_clean_records_if_needed) reconstructs the exact clean records from the securely stored source artifact in uploads_storage/, ensuring historical downloads remain 100% functional without mutating original data.

1.6 Production Cloud Architecture: Render + Neon + Neon Object Storage

To transition the platform from single-node local execution to a resilient, zero-cost cloud architecture while maintaining 100% local Windows development parity:



1. Compute Tier (Render Web Service):
- Hosts the FastAPI ASGI web application running on Uvicorn.
- Declaratively defined via render.yaml Blueprint specification, enabling automatic build and continuous deployment whenever changes land on branch main.
- Automated HTTPS/TLS termination, health monitoring probe at /api/health, and zero billing cost under Render's free web tier.
- Container filesystem is ephemeral: files written to disk are purged when instances spin down or redeploy. This necessitated decoupling stateful storage.
2. Persistence Tier (Neon Serverless PostgreSQL):
- Replaces local SQLite in production with a managed, serverless PostgreSQL database on Neon.tech.
- Dynamic connection string normalization in app/config.py converts legacy postgres:// URLs to SQLAlchemy-compliant postgresql:// and guarantees sslmode=require.
- Engine connection pooling (pool_size=5, max_overflow=10, pool_recycle=300, pool_pre_ping=True) handles Neon's serverless cold-start and idle connection termination gracefully.
- Relational integrity, constraints, indexes, and cascade operations are preserved identically between local SQLite and remote PostgreSQL.
3. Object Storage Tier (Neon Object Storage Persistent Storage):
- Stores raw uploaded CSV files and reprocessing artifacts using Neon Object Storage (S3-compatible API via boto3).
- Sits natively alongside the PostgreSQL database in the same Neon project and AWS region (Singapore / ap-southeast-1), minimizing cross-cloud latencies and eliminating external vendor credentials.
- Implemented via the StorageBackend abstraction in app/services/storage.py, supporting local filesystem storage (LocalStorageBackend) for offline development and NeonObjectStorageBackend (with backward-compatible S3StorageBackend and R2StorageBackend aliases) for production.
- Features automatic local caching, key sanitization against directory traversal, and idempotent clear operations.

4. Administrative & Public Reset Configuration:
   - The internal ALLOW_DESTRUCTIVE_RESET setting is computed from the environment variable FORCE_DISABLE_DESTRUCTIVE_RESET. The default value false leaves reset enabled for the assignment demo; setting FORCE_DISABLE_DESTRUCTIVE_RESET=true disables the reset endpoint.
   - In the current assignment-demo configuration, reset is enabled without requiring an administrator token or login so evaluators can verify the workflow.
   - The frontend uses a two-step confirmation: dragging the slider to the end unlocks the checkbox; ticking the checkbox enables the "Permanently Delete" button; ticking alone does not delete data; clicking the enabled button performs the reset and reloads the website (window.location.reload()).
   - Security Note: In this evaluation mode, direct calls to POST /api/dashboard/reset-data bypass the slider and checkbox because the backend does not require an administrator session or token. Do not use this configuration with sensitive production data.
   - A status endpoint (GET /api/dashboard/reset-status) reports the active reset policy and backend configuration.

2. ETL Workflow (Extract, Transform, Load)

Phase 1: Extract

1. Stream Intake: File bytes are received via POST /api/uploads.
2. Guardrails: File size is verified against UPLOAD_MAX_SIZE_BYTES (10 MB default). Files exceeding this limit trigger an immediate HTTP 413.
3. Extension & Format Verification: The filename is inspected for a .csv extension (HTTP 415 on mismatch).
4. Encoding Resolution: Detects UTF-8 with BOM (utf-8-sig), standard UTF-8, and falls back to ISO-8859-1 (latin-1) or CP1252.
5. Structural Parsing: The header row is extracted and normalized to lowercase. It checks for:
   - Empty/blank files (HTTP 400).
   - Duplicate column headers (e.g. ['name', 'email', 'name']).
   - Missing required columns for the targeted dataset.
6. Checksum Calculation: A SHA-256 hash of the uploaded raw bytes is computed and stored.

Phase 2: Transform

1. Data Normalization:
   - Whitespace: Strips leading/trailing spaces from all string fields.
   - Blank Normalization: Blank strings ("", " ", "NULL", "N/A") are converted to None.
   - Name Normalization: Collapses internal repeated whitespace into a single space and applies Title Case.
   - Email Normalization: Strips whitespace and lowercases the address.
   - Currency Normalization: Converts to uppercase and validates membership in {"USD", "EUR", "GBP", "CAD", "AUD", "JPY", "INR"}.
   - Date Normalization: Attempts parsing against 12 standard formats (ISO, US, European) and converts valid dates to canonical YYYY-MM-DD. Strict calendar validity is enforced (e.g., rejecting 2023-02-29 as non-leap-year).
   - Numeric Normalization: Strips currency formatting and parses finite numbers, explicitly rejecting NaN, +Inf, and -Inf. Prohibits negative amounts, quantities, and prices.
2. Error Collection:
   - Evaluates each record against all validation rules.
   - Preserves multiple error diagnostics per row (e.g., a row with both an invalid email and a missing date receives distinct error records).
   - Original un-sanitized values and raw row JSON are preserved for auditability.

Phase 3: Load

1. Transactional Integrity:
   - All valid clean records are staged for bulk insertion into the respective domain table (customers, sales, or inventory).
   - All validation errors are staged for insertion into validation_errors.
   - The job record in processing_jobs is updated with final accounting tallies, status completed, and duration.
   - A single db.commit() persists the batch atomically.
   - In the event of an unexpected database failure, db.rollback() executes, leaving zero partial clean records in the database, and the job status transitions to failed.

3. Idempotency & Duplicate Prevention Strategy

3.1 Stable Business Keys

Each dataset defines a natural or synthetic business primary key:
* Customers: customer_id (e.g. CUST-001)
* Sales: sale_id (e.g. SALE-1001)

* Inventory: product_id (e.g. PROD-5001)

3.2 In-File Deduplication

Within a single uploaded file, business keys are tracked in a transient hash set:
* The first valid record encountered is accepted.
* Any subsequent occurrence of the same key in the same file is flagged with error code DUPLICATE_KEY_IN_FILE and accounted for under duplicate_rows.

3.3 Database Deduplication

Before staging clean records, candidate business keys are batch-queried against the database:
* Keys already existing in clean tables are rejected with error code DUPLICATE_KEY_IN_DB and increment duplicate_rows.
* Clean tables maintain SQL PRIMARY KEY and UNIQUE constraints as a final defensive layer.

3.4 Repeated File Uploads

When an identical file (same SHA-256 hash) is uploaded:
* A new audit record is created in processing_jobs for end-to-end historical tracking.
* Pre-existing clean records in the database are recognized and accounted for as duplicates.
* The UI displays is_repeated_upload: true with a clear message: "File matches an existing upload... Processed with idempotency: 0 new, N duplicates."
* No records are duplicated or overwritten.

3.5 Accounting Invariant

To ensure mathematical integrity across all pipelines: Row accounting rule: Total rows = accepted rows + rejected rows + duplicate rows + skipped rows. Every discovered row belongs to one mutually exclusive category. An automated assertion enforces this equation before database commit.

3.6 In-File vs. Database Duplicate Distinction (Job #10 Case Study)

A real-world case study was audited in Job #10 (customers_mixed_quality.csv, 6 rows):

1. Source File Structure:
   - Line 2: CUST101 (Neha Verma, valid format)
   - Line 3: CUST102 (Missing email -> Rejected)
   - Line 4: CUST103 (Invalid email format -> Rejected)
   - Line 5: CUST104 (Missing name -> Rejected)
   - Line 6: CUST105 (Invalid date -> Rejected)
   - Line 7: CUST101 (Duplicate Customer, repeated ID)
2. Execution Against Database with Existing Records:
   - CUST101 was already committed to the clean database prior to Job #10.
   - When Line 2 was evaluated, it passed schema validation but matched an existing primary key in the database: categorized as DUPLICATE_KEY_IN_DB (Duplicate #1).
   - Lines 3 through 6 failed field validation: categorized as REJECTED (Rejected #1, #2, #3, #4).
   - When Line 7 was evaluated, it passed schema validation, but CUST101 had already appeared on Line 2 of this file: categorized as DUPLICATE_KEY_IN_FILE (Duplicate #2).
   - Result: Total: 6, Accepted: 0, Rejected: 4, Duplicates: 2, Skipped: 0.
   - Accounting check: 0 + 4 + 2 + 0 = 6.
   - Validation Error Records: Exactly 6 (1 for DB duplicate on row 2, 4 for rejected rows, 1 for in-file duplicate on row 7).
3. Execution Against an Empty Database:
   - If the same file is run against a fresh empty database, Line 2 is Accepted (inserted into clean tables).
   - Lines 3 through 6 remain Rejected (4).
   - Line 7 remains an in-file Duplicate (1).
   - Result: Total: 6, Accepted: 1, Rejected: 4, Duplicates: 1, Skipped: 0.
   - Accounting check: 1 + 4 + 1 + 0 = 6.
   - Validation Error Records: Exactly 5.

4. Testing Strategy & Execution Verification

4.1 Test Isolation via In-Memory SQLite

Tests are isolated from the developer's operational database:
* tests/conftest.py configures a shared in-memory SQLite database (sqlite:///:memory: with StaticPool).
* FastAPI's get_db dependency is overridden to inject the test database session.
* Database tables are dropped and recreated on every test function run.

4.2 Baseline Test Suite Results (Before CodeZero Follow-Up Patches)

The baseline test suite was executed locally using pytest before the CodeZero-review follow-up patches:

> Historical baseline: The 99-pass result below predates the follow-up fixes and regression tests. The latest post-fix
> run is recorded in Section 4.3.

```
============================= test session starts ==============================
platform win32 -- Python 3.12.5, pytest-9.1.1, pluggy-1.7.0
rootdir: C:\Users\rjha1\Downloads\csv_project
plugins: anyio-4.15.1, asyncio-1.4.0
collected 99 items

tests/test_api.py::test_api_health PASSED [ 1%]
tests/test_api.py::test_api_upload_success PASSED [ 3%]
tests/test_api.py::test_api_upload_unsupported_file_extension PASSED [ 4%]
tests/test_api.py::test_api_upload_empty_file PASSED [ 6%]
tests/test_api.py::test_api_upload_invalid_dataset PASSED [ 7%]
tests/test_api.py::test_api_jobs_list_and_detail PASSED [ 9%]
tests/test_api.py::test_api_job_not_found PASSED [ 10%]
tests/test_api.py::test_api_validation_errors_endpoints PASSED [ 12%]
tests/test_api.py::test_api_clean_records PASSED [ 13%]
tests/test_api.py::test_api_clean_records_unsupported_dataset PASSED [ 15%]
tests/test_api.py::test_api_dashboard_endpoints PASSED [ 16%]
tests/test_api.py::test_api_reprocess_endpoint PASSED [ 18%]
tests/test_api.py::test_download_clean_records_customers PASSED [ 20%]
tests/test_api.py::test_download_clean_records_sales_and_inventory PASSED [ 21%]
tests/test_api.py::test_download_clean_records_zero_accepted_returns_400 PASSED [ 23%]
tests/test_api.py::test_download_validation_errors_csv PASSED [ 24%]
tests/test_api.py::test_download_validation_errors_zero_errors_returns_400 PASSED [ 26%]
tests/test_api.py::test_download_job_summary_excel PASSED [ 27%]
tests/test_api.py::test_download_job_summary_excel_empty_clean_and_empty_errors PASSED [ 29%]
tests/test_api.py::test_download_job_summary_excel_sales_and_inventory PASSED [ 30%]
tests/test_api.py::test_download_job_summary_excel_isolation PASSED [ 32%]
tests/test_api.py::test_download_nonexistent_job_returns_404 PASSED [ 33%]
tests/test_api.py::test_download_job_isolation PASSED [ 35%]
tests/test_api.py::test_download_csv_escaping PASSED [ 36%]
tests/test_dashboard_upgrade.py::test_reset_preferences_endpoint PASSED [ 38%]
tests/test_dashboard_upgrade.py::test_dashboard_summary_and_trends_date_filtering PASSED [ 40%]
tests/test_etl.py::test_etl_valid_customers_loaded PASSED [ 41%]
tests/test_etl.py::test_etl_in_file_duplicate_detection PASSED [ 43%]
tests/test_etl.py::test_etl_existing_db_duplicate_prevention PASSED [ 44%]
tests/test_etl.py::test_etl_repeated_file_idempotency PASSED [ 46%]
tests/test_etl.py::test_etl_reprocess_job PASSED [ 47%]
tests/test_customers_mixed_quality_empty_db_vs_populated_db PASSED [ 49%]
tests/test_etl.py::test_row_not_classified_as_both_rejected_and_duplicate PASSED [ 50%]
tests/test_etl.py::test_multiple_errors_on_single_row_exceeds_rejected_count PASSED [ 52%]
tests/test_reset_and_filter.py::test_destructive_reset_endpoint_clears_all_tables PASSED [ 53%]
tests/test_reset_and_filter.py::test_destructive_reset_cleans_upload_artifacts PASSED [ 55%]
tests/test_reset_and_filter.py::test_destructive_reset_concurrency_lock PASSED [ 56%]
tests/test_reset_and_filter.py::test_reset_data_route_in_openapi_specification PASSED [ 58%]
tests/test_reset_and_filter.py::test_reset_data_idempotent_empty_database PASSED [ 60%]
tests/test_reset_and_filter.py::test_reset_data_preserves_files_outside_uploads_storage PASSED [ 61%]
tests/test_reset_and_filter.py::test_reset_data_transaction_rollback_on_failure PASSED [ 63%]
tests/test_reset_and_filter.py::test_records_endpoint_date_filtering PASSED [ 64%]
tests/test_reset_and_filter.py::test_errors_endpoint_date_filtering PASSED [ 66%]
tests/test_reset_and_filter.py::test_empty_date_range_strict_zeroes PASSED [ 67%]
tests/test_reset_and_filter.py::test_kpi_accounting_invariant_reconciliation PASSED [ 69%]
tests/test_transformation.py::test_clean_string PASSED [ 70%]
tests/test_transformation.py::test_normalize_name PASSED [ 72%]
tests/test_transformation.py::test_normalize_email PASSED [ 73%]
tests/test_transformation.py::test_normalize_currency PASSED [ 75%]
tests/test_transformation.py::test_parse_and_normalize_date PASSED [ 76%]
tests/test_transformation.py::test_parse_float PASSED [ 78%]
tests/test_transformation.py::test_parse_int PASSED [ 80%]
tests/test_validation.py::test_valid_customer_row PASSED [ 81%]
tests/test_validation.py::test_customer_missing_required_fields PASSED [ 83%]
tests/test_validation.py::test_customer_invalid_email PASSED [ 84%]
tests/test_validation.py::test_customer_invalid_date PASSED [ 86%]
tests/test_validation.py::test_valid_sale_row PASSED [ 87%]
tests/test_validation.py::test_sale_negative_amount PASSED [ 89%]
tests/test_validation.py::test_sale_unsupported_currency PASSED [ 90%]
tests/test_validation.py::test_valid_inventory_row PASSED [ 92%]
tests/test_validation.py::test_inventory_negative_quantity_and_price PASSED [ 93%]
tests/test_csv_import_empty_file PASSED [ 95%]
tests/test_validation.py::test_csv_import_missing_columns PASSED [ 96%]
tests/test_validation.py::test_csv_import_duplicate_headers PASSED [ 98%]
tests/test_validation.py::test_csv_import_utf8_bom PASSED [100%]

====================== 99 passed, 17 warnings in 48.52s ======================
```

4.3 Latest Post-Fix Test Results (October 26, 2026)

After fast-forwarding the local main branch to GitHub commit 1d45be9, the full suite was rerun using:

```
.\.venv\Scripts\python.exe -m pytest -q
```

* Passed: 109
* Failed: 0
* Warnings: 23 deprecation warnings from the framework/database stack
* Execution time: 19.91 seconds

This run includes the added regression coverage for job reprocessing/audit exports, trend-window selection, bounded upload reads, malformed CSV row widths and multiline row locations, and invalid date filters. Tests use in-memory SQLite; this result does not independently verify the production PostgreSQL connection or the deployed Render service.

5. Antigravity-Assisted Development & Review Process

> Tooling note: This document distinguishes the main implementation work assisted by Google Antigravity from the actual
> CodeZero exploration and independent review. The CodeZero review and its limitations are recorded in Section 5.4 and
> CODEZERO_REVIEW.md.

5.1 Tasks Assisted by Antigravity

1. Workspace Architecture Scaffolding: Designed the decoupled folder structure (app/, tests/, sample_data/, scripts/).
2. Database Modeling: Created SQLAlchemy 2.x models with cascade relationships, indexes, and primary keys.
3. Data Cleaning & Normalization Engine: Implemented comprehensive regex date parsing across multiple formats, string casing rules, and numerical validation.
4. ETL Pipeline & Transaction Boundaries: Authored the Extract-Transform-Load pipeline enforcing the mutual-exclusion accounting equation.
5. REST API Construction: Implemented OpenAPI-documented endpoints for health, ingestion, jobs, validation error itemization, clean records browsing, and dashboard trends.
6. Frontend Dashboard: Created a responsive, modern HTML5/CSS3/Vanilla JS single-page interface with Chart.js visualization.
7. Automated Testing Suite: The pre-review baseline had 99 test cases. Ten additional test cases were added in the CodeZero-review follow-up for reprocessing export preservation, bounded uploads, invalid date filters, malformed CSV widths, multiline row locations, and trend-window selection. The current collection is 109 test cases; the post-fix full-suite run passed all 109 tests (23 warnings in 19.91 seconds), as documented in Section 4.3.

5.2 Review & Manual Corrections Made During Development

During the implementation, several critical design details were reviewed and refined:

1. Pydantic Settings vs Dotenv: Switched from pydantic-settings to native python-dotenv + pydantic.BaseModel to avoid external package version incompatibilities.
2. UTF-8 BOM Header Parsing: Standard UTF-8 decoding can leave a leading zero-width no-break space (\ufeff) on the first header column. Implemented multi-tier decoding prioritizing utf-8-sig to cleanly strip the byte order mark.
3. Calendar Date Validation: Rather than accepting any string that matches a regex pattern, strict calendar parsing via datetime.strptime was enforced, properly rejecting impossible calendar days such as February 29th on non-leap years (2023-02-29) or month values above 12.
4. CSV Row Numbering (CodeZero follow-up): The parser now records the physical start line of each logical CSV record using csv.reader.line_num, including when prior quoted fields contain embedded newlines. A multiline-record regression test was added and passed in the post-fix full-suite run documented in Section 4.3.
5. Idempotent Job Reprocessing: Added a dedicated POST /api/jobs/{id}/reprocess endpoint that retrieves the saved file artifact and re-runs the pipeline safely without duplicating clean records.
6. Spreadsheet Formula Injection Mitigation: Sanitized cell values beginning with =, +, -, @, \t, \r by prepending a single quote in both CSV and multi-sheet Excel exports.

5.3 CodeZero Workflow & Tooling Demarcation

Prompt wording and iteration evidence are documented separately in CODEZERO_PROMPT_EVIDENCE.md <CODEZERO_PROMPT_EVIDENCE.md>. That file distinguishes reconstructed examples from verbatim prompt logs because the original transcript was not preserved.

This section documents the tools used in the actual development workflow for the assignment.

* CodeZero - initial exploration: Used to inspect the standalone starter exercise (csv_quality_checker.py), analyze baseline validation requirements, and plan the initial architecture and README scope (Git commit 7983368c3879efe141b90b7a269c1cb0882505b1, Plan 6adcf6109cb64fb3236a5352). That exercise file is not part of the final web-app repository.
* CodeZero - independent review and verification: Later reviewed the current source against the assignment, ran .\.venv\Scripts\python.exe -m pytest -q, inspected code paths behind identified risks, and ran a runtime probe that reproduced the reprocessing/audit-record issue. That baseline run reported 99 passing tests and 17 deprecation

warnings using in-memory SQLite. CodeZero's review session itself did not modify application files; a separate follow-up pass then applied targeted fixes and added regression tests. Those post-review changes were subsequently covered by a local full-suite run after updating to commit 1d45be9, which passed 109 tests with 23 deprecation warnings. See Section 4.3, Section 5.4, and CODEZERO_REVIEW.md <CODEZERO_REVIEW.md>.
* Google Antigravity: Assisted with the main full-stack implementation, including FastAPI routes, SQLAlchemy persistence, the ETL pipeline, the HTML/CSS/JavaScript dashboard, Excel/CSV reporting, and automated tests.

The initial findings came from the CodeZero review; a separate follow-up pass applied targeted code changes and added regression tests for the main issues. After updating local main to commit 1d45be9, the full suite passed 109 tests with 23 deprecation warnings in 19.91 seconds. This verifies the local code against the in-memory SQLite test fixture, not live PostgreSQL or the deployed service.

5.4 CodeZero Repository Review - October 26, 2026

The independent review is documented in CODEZERO_REVIEW.md <CODEZERO_REVIEW.md>. In the review session, CodeZero inspected the repository files, ran the local pytest suite, and reproduced one reprocessing/audit-record defect using a runtime probe. The recorded CodeZero baseline was 99 passed, 17 deprecation warnings. A separate post-fix local run after pulling commit 1d45be9 passed 109 tests with 23 deprecation warnings in 19.91 seconds. The tests use in-memory SQLite; neither run by itself verifies PostgreSQL or the deployed service.

The report identified issues involving reprocessing audit/export preservation, the trend chart's 30-job window, upload-size checking before a full read, malformed CSV row shapes and multiline row locations, invalid date-filter behavior, and documentation consistency for the SQL DDL and sample CSV inventory. Follow-up patches have since been applied: reprocessing now preserves the job's accepted-row snapshot, trends select the newest 30 jobs and display them chronologically, uploads read at most the configured limit plus one byte, malformed row widths are rejected while row start lines account for quoted multiline fields, and invalid/reversed date ranges return HTTP 422. Regression tests were added or updated for these cases. The review session itself made no application-code modifications; the post-patch suite was later run locally and passed all 109 tests.

6. What Was Verified & Remaining Limitations

6.1 Verified Capabilities

* [x] Application successfully compiles and boots on Windows with Uvicorn.
* [x] SQLite database tables are automatically initialized on startup.
* [x] Real multipart CSV uploads are parsed, decoded, and ingested.
* [x] Unsupported extensions (.txt) return HTTP 415.
* [x] Empty files return HTTP 400.
* [x] Missing required columns return descriptive validation failures.
* [x] Clean records are properly stored in database tables.
* [x] Validation errors are stored with row number, field name, error code, explanation, and original value.
* [x] In-file duplicate keys are rejected.
* [x] Database pre-existing duplicate keys are rejected.
* [x] Repeated uploads of identical files are audited without duplicating clean data.
* [x] Latest local post-fix test suite passed 109 tests with zero failures (23 deprecation warnings in 19.91 seconds) after updating to GitHub commit 1d45be9; see Section 4.3.
* [x] Interactive UI with KPI cards, Chart.js trends, jobs table, error logs, and records browser is fully connected to the live backend.
* [x] Job-scoped Clean Records (CSV) download endpoint (GET /api/jobs/{id}/download/clean) with correct headers, dataset columns, CSV escaping, and HTTP 400 when 0 accepted rows.
* [x] Job-scoped Validation Errors (CSV) download endpoint (GET /api/jobs/{id}/download/errors) with error tracking metadata, CSV escaping, and HTTP 400 when 0 errors.
* [x] Job-scoped Execution Summary Workbook (Excel .xlsx) download endpoint (GET /api/jobs/{id}/download/summary) with Job Summary, Clean Records, Validation Errors, frozen panes, filters, number formatting, and accounting reconciliation.
* [x] Complete UI integration in upload feedback card and Job Details modal with disabled states, tooltips, loading feedback, and double-click prevention.
* [x] Destructive Data Reset (POST /api/dashboard/reset-data) with concurrency lock (threading.Lock()), foreign-key safe cascade deletion, physical file unlinking with path traversal protection, transaction rollback, and schema preservation.
* [x] End-to-end date filtering across all 5 telemetry endpoints (/api/dashboard/summary, /api/dashboard/trends, /api/jobs, /api/records, /api/errors) with SQL joins and strict zero-state displays.
* [x] Data quality score and KPI row accounting invariant: Total Records = Accepted + Rejected + Duplicates + Skipped; the score is 0.0% (No data) when the total is 0.
* [x] Compact iPhone-style slide-to-reset control (210px x 42px), with a circular thumb, chevron icon, touch/mouse dragging, an 85% threshold, and a confirmation modal.
* [x] Dual themes (Warm Ivory & Dark Espresso) with WCAG AA compliant contrast on dataset select, drop zone, schema boxes, and disabled buttons.

* [x] Dynamic real-event notification store in localStorage tracking uploads, downloads, reprocessing, and resets with auto-hiding badge.

6.2 Remaining Limitations & Future Roadmaps

1. Background Job Queuing: Processing is currently synchronous. For files exceeding 100,000 rows, a message queue (e.g. Celery / RabbitMQ / Redis) should be introduced with SSE (Server-Sent Events) or WebSockets for live progress tracking.
2. User Authentication: Auth was intentionally omitted per assignment specifications. Production environments would require OAuth2/JWT authentication.
3. Complex Column Mapping: The platform currently expects standard normalized headers. A visual column-mapping wizard for mismatched legacy CSV headers could be added in a future milestone.

---

7. Destructive Data Reset Architecture (POST /api/dashboard/reset-data)

7.1 Objective & Safety Guarantees

In real data engineering operations, operators require the ability to purge staging or test data to reset the platform state without administrative intervention or dropping the database schema. The destructive data reset was engineered to meet the following non-negotiable safety criteria:

1. Zero Schema Modification: Does not drop tables, alter column definitions, or delete the SQLite database file (edvexa_data.db).
2. Atomic Transactional Rollback: All deletions execute inside an explicit SQLAlchemy transaction. If any stage fails, the entire transaction rolls back via db.rollback().
3. Concurrency Locking: Employs an in-process re-entrant lock (threading.Lock()) preventing concurrent upload or reset requests from causing race conditions or deadlocks.
4. Physical File Unlinking: Cleans up stored uploaded CSV files in settings.UPLOADS_DIR using path-traversal safeguards (file_path.resolve().is_relative_to(upload_dir_resolved)).
5. Separation from Preference Resets: POST /api/dashboard/reset-preferences resets client UI preferences (theme, filters), whereas POST /api/dashboard/reset-data performs the permanent data purge.

7.2 Foreign-Key Safe Deletion Order

SQLite enforces foreign key constraints. The deletion sequence must cascade from dependent child tables to parent entities:

```
# 1. Dependent clean records staged per job
deleted_records = db.query(JobCleanRecord).delete(synchronize_session=False)

# 2. Validation error itemizations linked to jobs
deleted_errors = db.query(ValidationError).delete(synchronize_session=False)

# 3. Domain business records
deleted_customers = db.query(Customer).delete(synchronize_session=False)
deleted_sales = db.query(Sale).delete(synchronize_session=False)
deleted_inventory = db.query(Inventory).delete(synchronize_session=False)

# 4. Parent processing jobs
deleted_jobs = db.query(ProcessingJob).delete(synchronize_session=False)

# 5. Commit batch atomically
db.commit()
```

7.3 API Response Contract

Upon successful execution, the endpoint returns an HTTP 200 payload detailing the exact counts of purged entities:

```
{
"status": "success",
"message": "All uploaded data, jobs, validation errors, clean records, and physical files have been permanently
cleared. Database schema preserved.",
"deleted_counts": {
"processing_jobs": 12,
"validation_errors": 48,
"job_clean_records": 34,
"customers": 15,
"sales": 10,
"inventory": 9,
"physical_files_removed": 12
},
"timestamp": "2026-10-25T04:20:00.000000"
}
```

7.4 Idempotent Empty-Database Handling

If an operator triggers the reset action when the database already contains 0 records, the API returns a graceful HTTP 200 rather than a 404 error:

```
{
```

```
"status": "success",
"message": "The application data is already empty.",
"deleted": {
"processing_jobs": 0,
"validation_errors": 0,
"job_clean_records": 0,
"customers": 0,
"sales": 0,
"inventory": 0,
"physical_files_removed": 0
},
"deleted_counts": {
"processing_jobs": 0,
"validation_errors": 0,
"job_clean_records": 0,
"customers": 0,
"sales": 0,
"inventory": 0,
"physical_files_removed": 0
},
"database_schema_preserved": true
}
```


The frontend slide-to-reset interaction detects this response and provides clear tactile feedback ("Already Empty"), avoiding confusing warning dialogues.

7.5 Process Reload & Diagnostic Resolution

During long-running local development sessions, the initial HTTP 404 observed on POST /api/dashboard/reset-data was diagnosed as a stale in-memory Uvicorn worker process that had been launched without --reload before the router endpoint was bound. The runner configuration was updated to always pass --reload during development, ensuring dynamic module synchronisation. Robust frontend fallback parsing was also implemented to safely catch non-JSON network responses.

---

8. End-to-End Date-Range Filtering Architecture

8.1 Unified Date Filtering Protocol

To support historical data auditing, all 5 primary telemetry endpoints accept optional start_date and end_date query parameters (in ISO format YYYY-MM-DD):

* GET /api/dashboard/summary?start_date=YYYY-MM-DD&end_date=YYYY-MM-DD
* GET /api/dashboard/trends?start_date=YYYY-MM-DD&end_date=YYYY-MM-DD
* GET /api/jobs?start_date=YYYY-MM-DD&end_date=YYYY-MM-DD
* GET /api/records?start_date=YYYY-MM-DD&end_date=YYYY-MM-DD
* GET /api/errors?start_date=YYYY-MM-DD&end_date=YYYY-MM-DD

8.2 Relational Joining for Records and Errors

Because JobCleanRecord and ValidationError represent atomic audit entries created during job execution, their timestamp filtering is joined against the parent ProcessingJob.started_at:

```
# Clean Records filtering
query = db.query(JobCleanRecord).join(ProcessingJob, JobCleanRecord.job_id == ProcessingJob.id)
if start_dt:
query = query.filter(ProcessingJob.started_at >= start_dt)
if end_dt:
query = query.filter(ProcessingJob.started_at <= end_dt)
```

```
# Validation Errors filtering
query = db.query(ValidationError).join(ProcessingJob, ValidationError.job_id == ProcessingJob.id)
if start_dt:
query = query.filter(ProcessingJob.started_at >= start_dt)
if end_dt:
query = query.filter(ProcessingJob.started_at <= end_dt)
```

8.3 Strict Empty-State Invariants

When a date range filter yields zero matching jobs:

1. Aggregations: All tallies evaluate strictly to 0 (Total Records = 0, Accepted = 0, Rejected = 0, Duplicates = 0, Skipped = 0).
2. Quality Rate: Returns 0.0% with a prominent No data badge rather than misleading percentage figures.
3. Donut & Trends Charts: Donut center displays 0 with greyed tracks; trend line drops to 0.
4. Honest Empty Table Notices:
    - Jobs Table: "No processing jobs found for this date range."
    - Errors Table: "No validation errors found for this date range."
    - Clean Records Table: "No clean {dataset} records found for this date range."

9. Data Quality Score & Row Accounting Invariants

9.1 Mathematical Accounting Invariant

Every row processed by the platform is deterministically classified into exactly one mutually exclusive bucket: Row accounting rule: Total records processed = accepted records + rejected records + duplicate records + skipped records. No row can be both rejected and counted as a duplicate. An assertion verifies this identity prior to every database commit.

9.2 Quality Score Formula

The platform calculates data quality score using the standard enterprise ratio: Quality Score formula: When total records processed is greater than zero, divide accepted records by total records processed and multiply by 100. When no records have been processed, show the application's no-data state instead of a historical comparison. Clicking the Data Quality Score KPI card opens an interactive methodology modal displaying the current values for accepted records, rejected records, duplicate keys, total rows, and the verified mathematical equation.

10. Dynamic Real-Time Event Notification Architecture

10.1 Real Event Store

The static dummy notification counter was replaced by a dynamic, event-driven notification store managed via App.notifications and synchronized with browser localStorage (edvexa_notifications):

* Events Tracked:
  - UPLOAD_COMPLETED: Logged when a CSV ingestion run completes, including dataset type, row count, accepted records, and error counts.
  - UPLOAD_FAILED: Logged when an upload fails validation or structural parsing.
  - EXPORT_DOWNLOADED: Logged when Clean CSV, Errors CSV, or Excel .xlsx reports are exported.
  - JOB_REPROCESSED: Logged when a historical job is re-executed.
  - DATA_RESET: Logged when destructive data reset is confirmed and executed.
* Auto-Hiding Unread Badge: If unread_count === 0, the #notificationBadge element receives the d-none class, cleanly removing the badge from the UI.
* User Actions: The notification menu provides "Mark all as read" and "Clear all" actions with live DOM updates.

11. Dual Themes, WCAG AA Contrast Compliance & iPhone-Style Slide-to-Reset

11.1 Palette & Typography Specifications

The interface implements two curated color palettes designed for data density and long-session visual comfort:

* Default Light Theme (Warm Ivory):
  - Surface Background: Warm Ivory #FAF7F1
  - Main Content Background: #FFFEFC
  - Sidebar Background: Soft Oatmeal #F4EEE4
  - Card Surfaces: Pure White #FFFFFF with warm border #EADECB
  - Typography: Primary Charcoal #1A1917, Muted Text #6B655B
  - Accent / Primary Action: Warm Deep Navy #1E2B37
  - Metric Pill: Subdued Gold #D4A373
* Dark Theme (Dark Espresso):
  - Surface Background: Deep Espresso #181512
  - Main Content Background: #1F1B16
  - Sidebar Background: #1E1A15
  - Card Surfaces: Roasted Walnut #28231D with border #3D352B
  - Typography: Off-White #EDE6DD, Muted Text #9C948A
  - Accent / Primary Action: Muted Amber Gold #C2955A

11.2 WCAG AA Form Contrast Hardening

Audit of dark-mode form controls revealed contrast deficiencies that were resolved to ensure WCAG AA compliance, with a contrast ratio of at least 4.5:1:

* Dataset Dropdown (<select> & <option>): Explicitly styled with dark background #28231D, border #3D352B, and text #EDE6DD, preventing OS-level white select popups.
* Drop Zone Area (.drop-zone): Styled with dark background #231E18, dashed border #504538, and readable instructions #EDE6DD.
* Schema Reference Cards (.schema-box): High-contrast header #EDE6DD, muted explanatory copy #9C948A, and highlighted

inline <code> tags #C2955A on dark backgrounds.
* Disabled State (#submitUploadBtn): Styled with high contrast #9C948A on #342D25 border #504538 rather than faded opacity.

11.3 iPhone-Style Compact Slide-to-Reset Control

To prevent accidental destructive resets while delivering an intuitive, tactile interaction:
* Dimensions: Compact track 210px x 42px (desktop), scaling responsively to 140px x 36px on mobile viewports 480px wide or narrower and 145px on ultra-compact devices 360px wide or narrower.
* Circular Thumb: 36px x 36px circular pill with directional chevron SVG icon, smooth gradient, and grab cursor.
* Safety Threshold: Requires dragging the thumb to at least 85% of track width to complete the gesture.
* Two-Step Interaction: Completing the slide opens the confirmation modal and unlocks the confirmation checkbox ("I understand that this action will permanently delete the selected uploaded data and associated processing history.").
* Explicit Button Activation: Ticking the checkbox enables the red "Permanently Delete" button; ticking the box alone never deletes data.
* Atomic Execution & Automatic Refresh: Clicking "Permanently Delete" executes POST /api/dashboard/reset-data. Upon backend confirmation, the frontend triggers window.location.reload(), performing an automatic full page refresh so all dashboard metrics, charts, and tables display the fresh updated state.
* Tactile Feedback: Completing the reset displays a loading spinner before smoothly returning to rest position.

11.4 Mobile Responsiveness & Slide-Out Drawer

* Full layout flexibility down to 320px viewport width (tested on 320px, 375px, 768px, 1024px, 1920px).
* Navigation collapses into a responsive slide-out drawer on screens narrower than 992px, with backdrop dimming and touch-to-close behavior.
* Header items smoothly adapt: sun/moon icon labels collapse below 768px, preserving space for the compact slider and notification bell without horizontal overflow.

11.5 Dark Theme Table Contrast Hardening & Status Badge Tokens

Processing history and audit tables (.data-table-warm) were upgraded with dedicated dark espresso tokens:
* Table Headers (<thead>): Deep charcoal-espresso #1A1612, uppercase warm sand labels #C4B7A5, and bottom separator #352E25.
* Table Cells (<td>): Warm cream typography #EDE5DA with clean bottom cell dividers #2C261F and subtle hover illumination #27211B.
* Job ID & Code Tokens: Pill wrappers #2C251C with warm amber text #D4AA70 and fine border #45392B.
* Status Badges: Distinct semantic tokens across both themes:
    - .badge-success: Deep emerald tint #152A20, emerald text #4ADE80, border #1F4A34.
    - .badge-danger: Crimson tint #2E1818, red text #F87171, border #522323.
    - .badge-warning: Amber tint #2D2314, warm amber #FBBF24, border #543E1B.
    - .badge-info: Deep slate #19232F, blue text #60A5FA, border #243A52.

12. Credits & Tools Used

* Project: EDVEXA Data Quality and ETL Monitoring Platform
* Author: Raghav Kumar

Development Tools & Genuine Engineering Contributions

1. Google Antigravity (Advanced Agentic Pair Programming):
    - Served as the primary engineering assistant for the complete end-to-end full-stack implementation, backend REST API routing, SQLAlchemy 2.x relational persistence, openpyxl multi-sheet Excel workbook generator, responsive HTML5/Vanilla JS frontend with dual themes (Warm Ivory & Dark Espresso), date-range filtering engine, iPhone-style slide-to-reset control, automated verification scripts, and the 109-test automated pytest suite (latest local run: all tests passed).
2. CodeZero:
    - Used for initial starter-material exploration and architecture planning, then for an independent repository review, a 99-pass/17-warning baseline test run on in-memory SQLite, and a runtime probe reproducing a reprocessing/audit-record defect. A separate follow-up pass applied targeted fixes and regression tests; a later local run after pulling commit 1d45be9 passed 109 tests with 23 warnings in 19.91 seconds. Full findings and limits are in CODEZERO_REVIEW.md <CODEZERO_REVIEW.md>.
3. Python 3.12, FastAPI & Uvicorn:
    - Core application runtime, asynchronous REST API transport layer, Pydantic v2 data validation, and OpenAPI documentation endpoints (/docs, /redoc).
4. SQLAlchemy 2.x & SQLite:
    - Relational database models with declarative mapping, cascade relationships, index definitions, and transactional boundary isolation.

5. openpyxl:
   - Dedicated multi-sheet Microsoft Excel reporting engine with automated column width calculations, cell formatting, frozen panes, and CWE-1236 spreadsheet injection mitigation.
6. pytest & httpx:
   - Comprehensive automated unit, integration, ETL, reset, date-filtering, storage abstraction, and API testing suite; latest local post-fix run: 109 passed, 0 failed, 23 deprecation warnings, with in-memory SQLite isolation.
7. Git & GitHub:
   - Version control, remote repository hosting, and structured commit history tracking.
8. Cloud Infrastructure (Render, Neon & Neon Object Storage):
   - Zero-cost production cloud hosting architecture utilizing Render (ASGI compute), Neon (serverless PostgreSQL), and Neon Object Storage (S3-compatible persistent object storage).

13. Cloud Deployment & External Service Setup Guide

This guide outlines the deployment procedure for EDVEXA on Render, backed by Neon Serverless PostgreSQL and Neon Object Storage, within the providers' free-tier limits.

13.1 Neon Serverless PostgreSQL Setup

1. Create Free Account: Sign up at neon.tech <https://neon.tech> using GitHub or Google SSO.
2. Create New Project:
   - Name: edvexa-production-db
   - Region: Select nearest AWS region (e.g., ap-southeast-1 / Singapore or us-east-2 / Ohio).
   - PostgreSQL Version: 16 or 17 (default).
3. Obtain Connection String:
   - On the Neon Dashboard, copy the connection string under Connection Details.
   - Select Pooled connection or direct connection string.
   - Example format: postgresql://<username>:<redacted>@<your-neon-host>/<database>?sslmode=require (format reference only; never a working credential)
   - The application automatically ensures postgresql:// protocol prefix and appends sslmode=require.

13.2 Neon Object Storage Setup

1. Navigate to Object Storage in Neon: In the edvexa-production-db project console, select Object Storage (or Buckets).
2. Create Bucket:
   - Click Create bucket.
   - Bucket name: edvexa-uploads (Access: private).
3. Generate S3 API Credentials:
   - Generate S3-compatible credentials with storage:read and storage:write scopes.
   - Securely record:
     - Access Key ID
     - Secret Access Key
     - S3 Endpoint URL (e.g., https://<branch>.storage.<cell>.<region>.aws.neon.tech)

13.3 Deploying to Render via GitHub Blueprint

1. Log in to Render: Go to render.com <https://render.com> and log in with GitHub.
2. Option A: Deploy via Blueprint (render.yaml - Recommended):
   - Click New + > Blueprint.
   - Connect repository https://github.com/renavaraghav/edvexa-data-quality-etl-monitor.
   - Render detects render.yaml automatically.
   - When prompted for non-synced environment variables, supply:
     - DATABASE_URL: Your Neon PostgreSQL connection string.
     - ADMIN_RESET_TOKEN: Generate a unique high-entropy value and set it only in Render's environment settings; do not copy a token from documentation or commit a real value.
     - STORAGE_BACKEND: neon
     - NEON_STORAGE_ENDPOINT: Your Neon Object Storage S3 endpoint URL.
     - NEON_STORAGE_BUCKET_NAME: edvexa-uploads
     - NEON_STORAGE_ACCESS_KEY_ID: Neon storage Access Key ID.
     - NEON_STORAGE_SECRET_ACCESS_KEY: Neon storage Secret Access Key.
   - Click Apply. Render will automatically build the service, run migrations/initialization on startup, and expose a live HTTPS URL.
3. Option B: Manual Web Service Creation:
   - Click New + > Web Service.
   - Connect repository https://github.com/renavaraghav/edvexa-data-quality-etl-monitor.
   - Runtime: Python 3.
   - Build Command: pip install --upgrade pip && pip install -r requirements.txt.

- Start Command: uvicorn app.main:app --host 0.0.0.0 --port $PORT.
- Health Check Path: /api/health.
- Plan: Free.
- Under Environment Variables, add the variables documented above.
- Click Create Web Service.

13.4 Environment Variables Reference

```
Variable Name | Required | Default | Description
ENVIRONMENT | Yes | development | Runtime mode: development or production. Governs security policies.
DATABASE_URL | Yes | sqlite:///... | Connection URI. PostgreSQL (postgresql://...) or SQLite.
STORAGE_BACKEND | Optional | local | local for filesystem storage or neon for Neon Object Storage.
NEON_STORAGE_ENDPOINT | If Neon | None | Neon S3-compatible endpoint URL.
NEON_STORAGE_BUCKET_NAME | If Neon | edvexa-uploads | Target bucket name.
NEON_STORAGE_ACCESS_KEY_ID | If Neon | None | S3 API Access Key ID.
NEON_STORAGE_SECRET_ACCESS_KEY | If Neon | None | S3 API Secret Access Key.
ADMIN_RESET_TOKEN | Optional | None | Secret token required to invoke POST /api/dashboard/reset-data when
administrative authorization is enforced.
FORCE_DISABLE_DESTRUCTIVE_RESET | Optional | false | Set to true to disable the destructive reset endpoint. When unset
or false, reset remains enabled for the assignment demo.
CORS_ORIGINS | Optional | * | Comma-delimited list of permitted CORS origins.
UPLOAD_MAX_SIZE_BYTES | Optional | 10485760 | Maximum file upload payload size (10 MB).
```

13.5 Free-Tier Sustainability & Operational Boundaries

* Render Web Service: In the free tier, Render automatically spins down services after 15 minutes of inactivity. When a new incoming request arrives, a cold start of 30-50 seconds occurs. Because database state is persisted on Neon and files are stored on Neon Object Storage, zero data is lost across spindowns.
* Neon PostgreSQL & Object Storage: Neon provides free-tier storage and compute hours more than sufficient for thousands of CSV processing jobs. Both relational records and raw uploaded files reside in the same project and region (Singapore / ap-southeast-1), minimizing network hops and avoiding third-party egress overhead.